

Full Length Article

Graph positive-unlabeled learning via Bootstrapping Label Disambiguation

Chunquan Liang^{a,b,*}, Luyue Wang^a, Xinyuan Feng^a, Yuying Cheng^a, Mei Li^{a,b},
Shirui Pan^c, Hongming Zhang^{a,b,*}

^a College of Information Engineering, Northwest A&F University, Shaanxi, China

^b Shaanxi Engineering Research Center for Intelligent Perception and Analysis of Agricultural Information, Shaanxi, China

^c School of ICT, Griffith University, Southport, QLD 4215, Australia

ARTICLE INFO

Keywords:

Positive-Unlabeled Learning
Label Disambiguation
Representation Learning

ABSTRACT

Graph positive-unlabeled learning is an important task that tries to learn binary classification models from only positive and unlabeled (PU) nodes. While state-of-the-art methods focus on training graph neural networks, they frequently rely on weak objective functions derived solely from a given class prior probability or inferred exclusively from the graph structure, leading their performance significantly lags behind that of fully labeled counterparts. In this paper, we fill this gap by treating unlabeled nodes as samples ambiguously labeled as both positive and negative, and by introducing a learning method called Bootstrap Label Disambiguation (BLD), which progressively resolves label ambiguities during the training of binary classifiers. BLD comprises a node representation learning module via bootstrapping and a novel central region-based label disambiguation strategy. The learning module leverages both previous representations and the derived positive centroid as targets to train positive-aligned representations, eliminating the need for a prior. Consequently, the disambiguation strategy constructs a central-region to identify ambiguous nodes and steadily transforms them into effective supervision. Extensive experiments on a range of real-world datasets show that our BLD method significantly outperforms existing approaches and in many cases even surpasses fully labeled classification models. The source code is available at <https://github.com/yunyun85/BLD>.

1. Introduction

Learning from positive and unlabeled (PU) data (Bekker & Davis, 2018) has been an important task over the past decades, as it is well suited for real-world applications where labeling data is costly and only positive and unlabeled samples are available. While traditional PU works have primarily concentrated on images (Chen et al., 2020; Zhao, Xu, Jiang, Wen, & Huang, 2022) and documents (Carnevali, Geraldini Rossi, Milios, & de Andrade Lopes, 2021; Liu, Dai, Li, Lee, & Yu, 2003; Liu & Peng, 2014), there is a growing trend towards exploring data organized in the form of graphs (Wu et al., 2019; Wu, Pan, Du, & Zhu, 2021; Zhao, Kong, & Yu, 2012). In PU learning with graph data, the objective is to train binary classifiers using PU nodes (Wu et al., 2021; Yang, Zhang, Yao, & Kwok, 2023; Yoo, Kim, Yoon, Kim, Jang, & Kang, 2022). This task has proven beneficial for numerous network applications. For instance, in paper recommendations within citation networks, only a few papers of interest are provided, and the goal is to predict whether the remaining papers are of interest or not. The provided examples serve as positive nodes, while the rest of the nodes in the network remain unlabeled. Similarly, in member identification

within social networks, only a few representative samples of an unknown group are known, and the objective is to determine whether other members also belong to that group. Here, the given samples are treated as positive, while the majority of nodes in the network are unlabeled. Similar examples could be also found in community detection within social networks (Kloumann & Kleinberg, 2014; Li et al., 2019) and spam sender monitoring (Janez-Martino, Alaiz-Rodriguez, Gonzalez-Castro, & Fidalgo, 2021).

Due to the practical significance, numerous graph PU learning methods have emerged (Wu et al., 2019, 2021; Yang et al., 2023; Yoo et al., 2022), with most of them proposing the training of Graph Neural Networks (GNNs) (Kipf & Welling, 2017; Veličković, Cucurull, Casanova, Romero, Lio, & Bengio, 2018) directly on PU nodes. Thanks to the powerful representation learning capability of GNNs (Duan, Lu, Wang, & Xuan, 2024; Duan, Xuan, Qiao, & Lu, 2024; Li et al., 2024), these methods have achieved state-of-the-art performance. Nonetheless, due to the lack of negatives, most of them require additional information such as class prior probabilities or graph structures to

* Corresponding author at: College of Information Engineering, Northwest A&F University, Shaanxi, China.
E-mail addresses: lcquan@nwsuaf.edu.cn (C. Liang), zhm@nwsuaf.edu.cn (H. Zhang).

<https://doi.org/10.1016/j.neunet.2025.107630>

Received 25 November 2024; Received in revised form 7 May 2025; Accepted 12 May 2025

Available online 31 May 2025

0893-6080/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.



Fig. 1. Illustration of current graph PU learning methods. Due to the lack of negative nodes, existing methods require additional information such as class prior probabilities or graph structures to unbiasedly estimate the prediction loss on PU nodes.

estimate the predicting loss like unbiased-risk estimators (Kiryo, Niu, Plessis, & Sugiyama, 2017; Plessis, Niu, & Sugiyama, 2015a, 2015b), marginals (Yoo et al., 2022), and distance-aware loss (Yang et al., 2023), as illustrated in Fig. 1. However, in practical applications, priors are often unknown, and estimating them is inherently challenging (Jain, Delano, Sharma, & Radivojac, 2020; Jain, White, & Radivojac, 2016). Additionally, graph structural information itself can be inaccurate, and more importantly, unbiased estimation inherently introduces errors. These cases result in the notably inferior performance of existing methods compared to fully labeled counterparts.

In this paper, we bridge the gap by formulating the graph PU learning as a partial label learning (PLL) problem (Lyu, Feng, Wang, Lang, & Li, 2021; Wang et al., 2022), where unlabeled nodes are considered instances ambiguously labeled as both positive and negative. Given the unknown class prior, we initialize each unlabeled node with a uniform class probability distribution as soft pseudo targets for binary classifier training. The primary challenge lies in label disambiguation. While existing PLL methods focus on disambiguation via learning discriminated representations, they all rely on partial labels to provide somewhat accurate supervisory information. However, in our scenario, the ambiguously labeled nodes with uniform pseudo targets are unable to offer meaningful guidance, and using solely positive nodes does not effectively separate negatives from positives in deep feature space. Furthermore, most existing PLL methods employ hard disambiguation, where ambiguous nodes are assigned exclusively to one class and remain unchanged. Nevertheless, learning optimal representations is challenging; therefore, this strategy is prone to introducing label errors into the final classifiers.

To this end, we propose a learning method called Bootstrapping Label Disambiguation (BLD) that progressively resolves label ambiguity and simultaneously trains a binary classifier. Our BLD method comprises a node representation learning module based on bootstrapping and a central region-based label disambiguation algorithm. The learning module leverages previous version of representations and the induced positive centroid as targets, and learn enhanced representations by minimizing the prediction losses directly on all nodes and P set within the deep feature space. It eliminates the need for a class prior, but aligns representations closely for positive nodes. Subsequently, the disambiguation algorithm builds a dependable region based on the derived centroid for progressively updating the soft pseudo targets of ambiguous nodes, thereby providing reliable and robust supervision for training binary classifiers.

Our contributions are mainly summarized as follows:

- (1) We formulate graph PU learning as a PLL problem and introduce the BLD method, which steadily resolves label ambiguity for ambiguous labeled nodes during the training of a binary classifier.
- (2) We introduce a novel label disambiguation strategy that consisting of a representation learning module based on bootstrapping and a central region-based disambiguation method.
- (3) We evaluate the effectiveness our BLD through empirical analysis of various real-world datasets. The experimental results show that our proposed BLD performs better than state-of-the-art methods, and in most cases, even outperforms supervised GNN algorithms.

2. Related works

PU learning. Plenty of PU learning methods have been proposed for various data (Bekker & Davis, 2018; Chen et al., 2020; He, Zhang, Li, & Shi, 2012; Zhao, Xu et al., 2022), mainly divided into three categories. (1) Class prior-based approach. It uses a given class prior probability to aid the training of conventional classifiers, like Bayesian models (He et al., 2012) or decision trees (Letouzey, 2005), for PU data. In particular, the prior can be used to estimate unbiased losses on PU samples (Kiryo et al., 2017; Plessis et al., 2015a, 2015b), with such loss, neural network models can be trained in end-to-end manner for classification. Unfortunately, the prior is often unknown in practices and poses a challenge for estimating it (Perini, Vercruyssen, & Davis, 2020; Zeiberg, Jain, & Radivojac, 2020). (2) Weight assignment method. This method assigns weights to unlabeled samples adjust them during training (Claesen, De Smet, Suykens, & De Moor, 2015; Mordelet & Vert, 2014; Zhu et al., 2023). It converts unlabeled samples into negatives with assigned weights and then utilizes conventional supervised or semi-supervised algorithms for the learning process. (3) Two-step strategy. The method assumes that the data are smooth and separable (Bekker & Davis, 2018; Liu et al., 2003; Liu & Peng, 2014) and implements with two key stages: firstly, selecting a group of reliable negative samples based on some distance metrics; and secondly, training conventional classifiers.

Graph data PU learning. Due to the practical importance in networks data, graph PU learning has also been widely studied. Some studies focus on graph-level classification (Wu, Pan, Zhu, Zhang, & Wu, 2016; Zhao et al., 2012), while the majority concentrate on node-level. Existing methods can generally be categorized into three groups: (1) Utilizing traditional or introducing novel GNNs (Duan, Lu et al., 2024; Duan, Xuan et al., 2024; Li et al., 2024) and pairing them with unbiased loss functions, such as GCN (Kipf & Welling, 2017), GAT (Veličković et al., 2018), and LSDAN (Wu, Pan, Du et al., 2019; Wu et al., 2021); (2) Introducing novel loss functions, such as structure-aware (SA) loss (Yang et al., 2023); (3) proposing a comprehensive learning framework for deriving useful supervision, such as GRAB (Yoo et al., 2022) and BLD (Liang et al., 2024).

In contrast to existing methods, this paper focuses on learning positive-aligned representations and progressively updating soft pseudo targets for unlabeled nodes to provide reliable and stable supervisory information for binary classifier learning.

Partial label learning. Partial label learning aims to learn from ambiguously labeled data where each training example is associated with a set of possible labels instead of a single ground-truth. The primary challenge lies in label disambiguation. Existing methods mainly include average based methods (Cour, Sapp, & Taskar, 2011; Zhang & Yu, 2015), maximum margin-based methods (Wang et al., 2020), graph-based methods (Lyu et al., 2021; Xu, Lv, & Geng, 2019), clustering-based approaches (Liu & Dietterich, 2012), self-training methods (Lv et al., 2020; Wen, Cui, Hang, Liu, Wang, & Lin, 2021), and contrastive learning methods (Wang, Xiao et al., 2022). While the majority of these approaches rely on partial labels to offer moderately precise supervisory information, the partially labeled nodes in our PLL setting cannot provide effective guidance. Therefore, current disambiguation strategies are not well-suited for our problem.

Some studies have attempted to model the PU learning as a PLL problem (Zhang, Ren, Liu, Yang, & Gong, 2019) by treating unlabeled

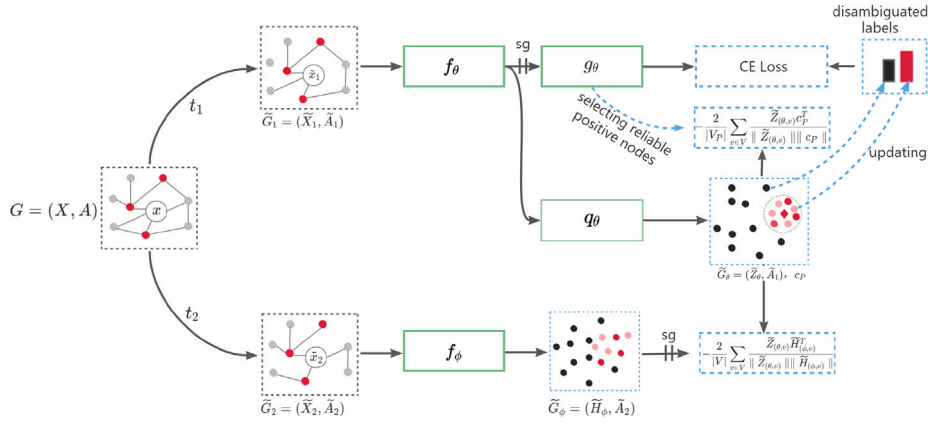


Fig. 2. Illustration of BLD framework. Taken two augmentations $\tilde{G}_1$ and $\tilde{G}_2$ of a graph G as input, the main network (comprising a GNN encoder f_θ and an MLP predictor g_θ) and the auxiliary network (including a GNN encoder f_ϕ) jointly learn node representations $\tilde{H}_\theta$ by conducting two bootstrapped learning tasks respectively on the P set and U set. The representation $\tilde{Z}_\theta$ and positive prototypes c_p are used to construct a reliable region to aid label disambiguation. Disambiguated labels are then used to train the classifier g_θ . Additionally, as an alternative, the most reliable positive nodes identified by classifier g_θ from the unlabeled nodes can be added to set P to participate in representation learning. Here θ and ϕ are learnable parameters, and **sg** means stop-gradient.

data as samples ambiguously labeled as both positive and negative. This approach makes the PLL methods mentioned above as potential solutions to the PU problem. However, they mainly focus on image data and used traditional maximum margin-based methods for hard label-disambiguation, which are also not suitable for our problem.

Bootstrapped representation learning. Bootstrapping representation learning enhances representations by using previous outputs as targets. Current methods fall into two categories: centroid-supervised and self-supervised. In centroid-supervised learning, class or cluster centroids derived from prior representations serve as targets, effectively capturing semantic similarities and excelling in tasks like face recognition (Wen, Zhang, Li, & Qiao, 2016), ambiguous class handling (Wang, Xiao et al., 2022), visual clustering (Li, Zhou, Xiong, Socher, & Hoi, 2021), and image segmentation (Zhao et al., 2022). In self-supervised learning, previous representations act as targets, capturing low-level semantic similarity and uniformity (Wang & Isola, 2020). This mechanism, inspired by reinforcement learning (Guo et al., 2020), was first applied to image representation learning in BYOL (Grill et al., 2020) and later adapted for graph data in BGRL (Thakoor et al., 2022). However, these methods are designed for fully supervised or self-supervised settings, making them unsuitable for the PU problem. To address this, we propose a novel approach combining centroid-supervised and self-supervised bootstrapping learning tailored for PU problems. This unique integration produces discriminative representations aligned with positive samples, enabling accurate label identification for unlabeled nodes and the construction of a robust binary classifier.

Few-shot learning. Similar to PU learning, few-shot learning (FSL) (Wang, Ding, Zhang, Chen, & Li, 2022; Wang, Huang, Li, Huang, Wu, & Wu, 2024; Wang, Tan, Liu, & Li, 2023) is a semi-supervised learning scenario that uses abundant unlabeled data but includes a small number of labeled samples (typically few per category). The core challenge of FSL is learning effective representations or leveraging prior knowledge to adapt quickly to new tasks with limited labeled data. In contrast, PU learning (PUL) involves only positive and unlabeled samples, with no explicit negative examples. The key challenge in PUL is identifying potential negatives or directly leveraging unlabeled data for discriminative learning. Due to these distinct problem settings, FSL methods cannot be directly applied to PU problems.

3. Method

In this section, we formulate graph PU learning as a PLL problem. We then introduce the Bootstrapping Label Disambiguation (BLD)

framework, which comprise a representation learning module via bootstrapping and a central region-based label disambiguation strategy. This framework supports the learning of useful representations and the progressive transformation of ambiguous targets into effective supervision during the train of binary classifiers.

3.1. Problem statement

We consider a graph $G = (V, X, A)$, where V is the set of n nodes, $X \in \mathbb{R}^{n \times d}$ denotes the feature matrix for all nodes in V , $A \in \mathbb{R}^{n \times n}$ represents the adjacency matrix, and d is the dimension of node features. We focus on node-level classification task. Given a graph G and a small set of positive nodes $V_P \subset V$, the goal is to train a binary classifier that predicts whether an unknown node $v \in V_U = V \setminus V_P$ should be labeled positive or negative.

3.2. Graph PU learning as a PLL problem

We model graph PU learning as a PLL problem by treating unlabeled nodes as samples ambiguously labeled as both positive and negative. Specifically, for each unlabeled node $v_i \in V_U$, given the unknown class prior, we initialize a uniform distribution $p_i = [0.5, 0.5]$ as its soft pseudo target. To unify the computation of prediction loss on the P set, we also assign each positive node $v_i \in P$ a vector $p_i = [0.0, 1.0]$ as its soft target. Our goal is to learn a GNN encoder f_θ , along with the subsequent binary classifier g_θ that comprises a Multi-Layer Perceptron (MLP) with a softmax output layer, where θ is a learnable parameters. Let $H_\theta = f_\theta(G)$ be the node representations. Similar to existing graph PU methods (Wu, Pan, Du et al., 2019; Wu et al., 2021; Yang et al., 2023), we select a subset $V_T \subset V_U$, in addition to V_P , to train the binary classifier. Specifically, we minimize Cross-Entropy (CE) loss, which measures the discrepancy between the predicted probability distribution of the classifier g_θ and the true (or estimated) probability distribution p_i . The overall prediction loss is calculated as follows:

$$\mathcal{L}_C(g_\theta; V_P, V_T) = - \sum_{v_i \in V_P} \log(g_\theta^1(H_{(\theta,i)})) - \sum_{v_i \in V_T} \sum_{j=0}^1 p_{i,j} \log(g_\theta^j(H_{(\theta,i)})) \quad (1)$$

where $H_{(\theta,i)}$ is the output representation of v_i given by f_θ , j denotes the indices of labels, $p_{i,j}$ denotes the pseudo target corresponding to the j -th label for input v_i , $g_\theta(\cdot)$ is the output of the MLP-softmax classifier, and $g_\theta^j(\cdot)$ refers to its j -th component of this output.

Obviously, the uniform soft pseudo target $[0.5, 0.5]$ for ambiguously labeled node cannot provide useful supervision for training the binary classifier g_θ , as well as the encoder f_θ . Therefore, the primary challenge

lies in label disambiguation. In the next section, we will offer a detailed explanation of our BLD framework for addressing this challenge, as illustrated in Fig. 2.

3.3. Bootstrapped representation learning for label disambiguation

Due to the lack of negative nodes, our goal is to learn closely-aligned representations exclusively for positive class. The primary challenges arise from two aspects: ambiguously labeled nodes with uniform pseudo targets are unable provide valuable supervision for representation learning either, and solely using positive nodes may fail to separate negatives from positives in the representation space, ultimately risking the model collapsing to a trivial solution.

Inspired by recent advancements in bootstrapped self-supervised representation learning (Grill et al., 2020; Guo et al., 2020; Thakoor et al., 2022) and centriod-supervised learning (Li et al., 2021; Wen et al., 2016), our key to achieve the goal is to integrate centriod-supervised learning with self-supervised learning. The former can capture the semantic similarity among positive nodes, bringing them closer, while the latter can capture the low-level semantic similarity and uniformity of each node, assisting the former and preventing model from collapsing.

Network Architecture. As mentioned above, we aim to learn a GNN encoder f_θ that generates node embeddings $H_\theta = f_\theta(X, A)$, followed by a MLP-softmax classifier g_θ for classifying unlabeled node. In order to integrate bootstrapped self-supervised learning task into centriod-supervised learning task to aid training of f_θ , we employ an auxiliary network similar to BGRL (Thakoor et al., 2022). Fig. 2 provides the overview of our BLD architecture. It comprises a primary network consisting of the GNN encoder f_θ , followed by a MLP predictor q_θ , and an auxiliary network containing a GNN encoder f_ϕ , with trainable parameters θ and ϕ .

Forward model. To conduct the self-supervised learning task, BLD creates two augmented views $(\tilde{X}_1, \tilde{A}_1)$ and $(\tilde{X}_2, \tilde{A}_2)$ of the graph (X, A) using two augmentation functions (Thakoor et al., 2022; You et al., 2020) t_1 and t_2 , respectively. These views are then inputted into f_θ and f_ϕ to generate main representations $\tilde{H}_\theta$ and auxiliary representations $\tilde{H}_\phi$, respectively. Subsequently, $\tilde{H}_\theta$ is fed to q_θ , generating predictions $\tilde{Z}_\theta$, thereby approximating the positive centroid.

Training objective. BLD trains the primary network by minimizing centriod-supervised learning loss on positive set V_P and self-supervised learning loss on all nodes V .

(1) Centriod-supervised representation learning. To estimate central loss $\mathcal{L}_P(\theta, \phi)$ on the P set, BLD first calculates the positive centroid c_P by taking the average of the predictions $\tilde{Z}_{(\theta,v)}$ over this set, as follows:

$$c_P = \frac{1}{|V_P|} \sum_{v \in V_P} \tilde{Z}_{(\theta,v)}. \quad (2)$$

BLD then trains the primary network using an augmented version $(\tilde{X}_1, \tilde{A}_1)$ to minimize the distance between the prediction and the centroid c_P . This encourages the positive examples to be pulled closer to the positive centroid in the deep feature space, enhancing the clustering of positive nodes. Specifically, we use cosine distance as a measure, which is advantageous because it is normalized and focuses on the orientation rather than the magnitude of the vectors, making it more robust to variations in feature scales. The central loss is calculated as follows:

$$\mathcal{L}_P(\theta, \phi) = -\frac{2}{|V_P|} \sum_{v \in V_P} \frac{\tilde{Z}_{(\theta,v)} c_P^T}{\|\tilde{Z}_{(\theta,v)}\| \|c_P\|}. \quad (3)$$

(2) Bootstrapped self-supervised representation learning. To compute self-supervised loss $\mathcal{L}_V(\theta, \phi)$ on all nodes V , BLD utilizes both augmented versions. Using one augmented version $(\tilde{X}_1, \tilde{A}_1)$, BLD trains

the primary network to minimize the distance between its predictions and the auxiliary representations derived from the other version $(\tilde{X}_2, \tilde{A}_2)$, which capture the low-level semantic similarity and uniformity of each node. Similarly, we employ cosine distance as the measure and compute self-supervised loss as follows:

$$\mathcal{L}_V(\theta, \phi) = -\frac{2}{V} \sum_{v \in V} \frac{\tilde{Z}_{(\theta,v)} \tilde{H}_{(\phi,v)}^T}{\|\tilde{Z}_{(\theta,v)}\| \|\tilde{H}_{(\phi,v)}\|}. \quad (4)$$

(3) Joint learning. We integrate the two bootstrapped representation learning tasks with the classification learning task to jointly train the primary network and the binary classifier. The overall loss can be computed as follows:

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_P(\theta, \phi) + \mathcal{L}_V(\theta, \phi) + \mathcal{L}_C(\theta, \phi). \quad (5)$$

(4) Updating Parameters of Networks. As with most deep learning approaches, we update the primary network f_θ and the classifier g_θ using stochastic gradient descent. Besides, similar to bootstrapped self-supervised representation learning (Grill et al., 2020; Thakoor et al., 2022), to prevent model collapse and create a bootstrapping effect that captures the low-level semantics of nodes, we update the auxiliary encoder f_ϕ using a slow-moving average of the main parameters θ , as follows:

$$\theta = \text{opt}(\theta, \eta, \nabla_\theta \mathcal{L}(\theta, \phi)), \quad (6)$$

$$\phi = \tau \phi + (1 - \tau) \theta. \quad (7)$$

Here, η and τ represent the learning rate and updating rate for ϕ , respectively.

Upon convergence, we retain only f_θ and g_θ for classifying unlabeled nodes. Nevertheless, training the binary classifier g_θ still requires precise supervision, a task made challenging by label ambiguity. To tackle this problem, we introduce a central region-based label disambiguation mechanism.

3.4. Central region-based label disambiguation

As mentioned earlier, the combination of the central loss on V_P and self-supervised loss on V creates a clustering effect in deep feature space where positive nodes tend to approach their centroid, while negatives tend to be uniformly distributed. Therefore, our disambiguation strategy is straightforward: an ambiguous node near the positive centroid is updated as positive, and conversely. However, this presents a challenge as we must establish a boundary to determine the proximity to the centroid required for a node to be considered a positive example for updating. To address this, we suggest defining a reliable region around the positive centroid, with a radius equal to the distance from the centroid to the farthest given positive nodes. During training, unlabeled nodes within this region are updated as positive, whereas those outside it are updated as negative.

To implement above idea, we calculate the cosine distance between each node and the positive centroid c_P in the deep feature space, which serves as the distance metric for each node, as follows:

$$d_\theta = 1 - \frac{\tilde{Z}_\theta c_P^T}{\|\tilde{Z}_\theta\| \|c_P\|}. \quad (8)$$

Additionally, we determine the boundary d_b by selecting the distance from the centroid to the farthest labeled positive node, as follows. This ensures that most positive nodes (whether labeled or unlabeled) will fall within the region formed by the boundary and the centroid.

$$d_b = \max_{v \in V_P} d_{(\theta,v)}, \quad (9)$$

where $d_{(\theta,v)}$ represents the distance from a node v to positive centroid c_P .

Using the estimated distance of each node to centroid and the boundary, we propose a softened and slow moving-average strategy to update the pseudo-targets for ambiguous (unlabeled) node $v_i \in V_T$, as follows:

$$p_i = \text{Normalize}(\gamma p_i + (1 - \gamma)e), \quad (10)$$

where γ is a target decay rate, and $e = [0, 1]$ if $d_{(\theta, v)} < d_b$ else $e = [1, 0]$. This progressively updating strategy offers reliable and robust supervision for training binary classifiers, as discussed in next section, and verified in experiments in Section 4.3.

The pseudo-code of BLD is given by Algorithm 1.

Algorithm 1: Pseudo-code of BLD

Input:
a graph $G = (V, X, A)$,
the set of positive nodes V_P ,
the set of unlabeled nodes V_U ,
augmentation functions t_1, t_2 ,
main parameters θ , encoder f_θ , classifier g_θ , predictor q_θ ,
auxiliary parameters ϕ and encoder f_ϕ ,
decay rate γ for updating pseudo target,
decay rate τ for updating auxiliary network,
optimizer opt for updating parameters,
epoch E .
Output: Encoder f_θ and MLP-softmax classifier g_θ

- 1 Select $V_T \subset V_U$; // Sample a set of unlabeled nodes
- 2 **for** $i = 1$ **to** E **do**
- 3 $\tilde{H}_\theta = f_\theta(t_1(X, A))$; // Compute main representations
- 4 $\tilde{H}_\phi = f_\phi(t_2(X, A))$; // Compute auxiliary representations
- 5 $\tilde{z}_\theta = q_\theta(\tilde{H}_\theta)$; // Obtain predictions
- 6 $c_p = \frac{1}{|V_P|} \sum_{v \in V_P} \tilde{z}_{(\theta, v)}$; // Calculate positive-centroid
- 7 $\mathcal{L}_P(\theta, \phi) = -\frac{2}{|V_P|} \sum_{v \in V_P} \frac{\tilde{H}_{(\theta, v)} c_p^T}{\|\tilde{H}_{(\theta, v)}\| \|c_p\|}$; // Central loss
- 8 $\mathcal{L}_V(\theta, \phi) = -\frac{2}{|V|} \sum_{v \in V} \frac{\tilde{z}_{(\theta, v)} \tilde{H}_{(\phi, v)}^T}{\|\tilde{z}_{(\theta, v)}\| \|\tilde{H}_{(\phi, v)}\|}$; // Self-supervised loss
- 9 $d_b = \max_{v \in V_P} d_{(\theta, v)}$; // Compute a boundary
- 10 **for** $j = 1$ **to** $|V_T|$ **do**
- 11 $e = [0, 1]$ if $d_{(\theta, j)} < d_b$ else $e = [1, 0]$
 $p_j = \text{Normalize}(\gamma p_j + (1 - \gamma)e)$; // Update soft targets
- 12 **end**
- 13 $\mathcal{L}_C(g_\theta; V_P, V_T) = -\sum_{v_i \in V_P} \log(g_\theta^1(\tilde{H}_{(\theta, i)})) - \sum_{v_i \in V_T} \sum_{j=0}^1 p_{i,j} \log(g_\theta^j(\tilde{H}_{(\theta, i)}))$; // Classification loss
- 14 $\mathcal{L}(\theta, \phi) = \mathcal{L}_P(\theta, \phi) + \mathcal{L}_V(\theta, \phi) + \mathcal{L}_C(g_\theta; V_P, V_T)$; // Total loss
- 15 $\theta = opt(\theta, \eta, \nabla_\theta(\mathcal{L}(\theta, \phi)))$; // Update θ
- 16 $\phi = \tau \phi + (1 - \tau)\theta$; // Update ϕ
- 17 **end**
- 18 Return f_θ and g_θ

3.5. Discussion

Complexity Analysis. In traditional GNN architectures like GCN (Kipf & Welling, 2017) and GAT (Veličković et al., 2018), node representations in a graph G with $n = |V|$ nodes and m edges are computed with a time and space complexity of $O(n + m)$. Our method, BLD, starts with two augmentation operations t_1 and t_2 , which randomly remove a given fraction of node attribute values and edges, achieved in $O(n + m)$

time and space. Subsequently, two GNN encoding operations, f_θ and f_ϕ , are conducted, also with a time and space complexity of $O(n + m)$. Additional operations, such as prediction q_θ , classification g_θ , centroid estimation c_p , updating pseudo targets, and various loss computations (including central loss $\mathcal{L}_P(\theta, \phi)$, self-supervised loss $\mathcal{L}_V(\theta, \phi)$, and classification loss $\mathcal{L}_C(\theta, \phi)$), can all be performed in $O(n)$ time and space. The complexity of backpropagation matches that of the forward pass. Consequently, the total time complexity and space complexity of the update step are $O(n + m)$.

Aligned representations for positive class. The central loss functions by penalizing the distance between the positive centroid and positive nodes, which in turn brings potential positive nodes that are similar to the given positives closer together in deep feature space. Essentially, centroid-supervised learning focuses on capturing positive semantic similarity. In contrast, self-supervised learning works to minimize the distance between the two views of graph nodes, aiming to capture both the inherent semantic similarity and uniformity of each node. By integrating self-supervised learning with centroid-supervised learning, positive similarity is strengthened, while also ensuring that negative nodes are uniformly dispersed according to their own distributions, effectively creating a distinction from positive nodes.

Conversely, despite not incorporating negative examples, the BLD model avoids collapsing into trivial solutions. This is attributed to the fact that the loss in self-supervised learning, due to the slow-moving-average updating to the auxiliary encoder, never converges to zero. Consequently, the joint learning of the both tasks safeguards against model collapse.

Progressive label disambiguation. Because node representations evolve continuously throughout the training process, the boundaries of regions for label disambiguation are also constantly changing. As a result, determining the optimal boundary for hard label-disambiguation—assigning ambiguously labeled nodes to either the positive or negative class exclusively, but not both—becomes challenging. Instead of hard label-disambiguation, we progressively update soft pseudo targets of ambiguously labeled nodes in each training epoch, and meanwhile utilize them to guide the training of the binary classifier. This strategy enhances the robustness of disambiguation and reduces the impact of inaccuracies in supervised training classifications that can arise from abrupt updates to hard labels. It is particularly beneficial for ambiguously labeled nodes near boundaries that may switch between interior and exterior of the region, ensuring that their soft pseudo targets better reflect their actual positions in deep feature space. Ultimately, it establishes a resilient decision boundary for binary classification.

4. Experimental study

In this section, we present results from an extensive empirical study across various real-life datasets to verify the effectiveness of our BLD method.

4.1. Experimental setup

(1) Baselines. We compare our BLD with baselines implemented using diverse strategies, including novel learning frameworks such as GRAB (Yoo et al., 2022) and BLP (Liang et al., 2024), diverse GNNs with unbiased loss like GCN-PU (Kipf & Welling, 2017), GAT-PU (Veličković et al., 2018), and LSDAN (Wu, Pan, Du et al., 2019; Wu et al., 2021), novel supervised loss with GCN such as SA-GCN (Yang et al., 2023), Asy-Loss (Wang, Pu, Xu, & Zhang, 2021), and Self-Correct (Zhu et al., 2023), label propagation methods such as LABEL-PPG (Carnevali et al., 2021), sampling unlabeled nodes as negatives like GCN-UN-PU, and PLL methods like PiCO (Wang, Xiao et al., 2022).

Below is a detailed introduction of the baselines.

GRAB (Yoo et al., 2022). The Graph-based Risk Minimization with Iterative Belief Propagation (GRAB) method is an iterative technique that calculates marginals for each unlabeled node by treating the

provided graph as a Markov network and incorporating the estimated class prior. By integrating the computed marginals into the objective function, it trains a binary classifier, which is then used to refine the class prior estimation. In our implementation, we supply GRAB with the actual class prior instead of estimating it, allowing GRAB to run just once.

BLP (Liang et al., 2024). The Bootstrap Latent Prototypes (BLP) framework consists of a graph representation learning module and a two-step strategy algorithm. The learning module generates valuable representations by predicting latent prototypes, while the algorithm module leverages these representations to identify trustworthy negative nodes and train a binary classifier.

GCN (Kipf & Welling, 2017). The Graph Convolutional Network (GCN) employs multiple graph convolutional layers to combine the node's representation with that of its neighbors, updating the node's representation. To enable direct PU learning with GCN, we apply the non-negative unbiased risk estimator from (Kiryo et al., 2017) to train the GCN, naming this model GCN-PU.

GAT (Veličković et al., 2018). The Graph Attention Network (GAT) utilizes a self-attention mechanism, enabling each node to focus on its neighbors with varying attention weights and adjusting the representation for each node accordingly. Like with GCN, we use the non-negative unbiased risk estimator (Kiryo et al., 2017) to train the GAT and call this model GAT-PU.

LSDAN (Wu, Pan, Du et al., 2019; Wu et al., 2021). The Long-Short Distance Aggregation Networks (LSDAN) generate multiple graphs at different distances using the adjacency matrix. They then incorporate a long-short distance attention mechanism to update node representations based on these graphs. LSDAN is more intricate than GAT because it applies the attention mechanism to nodes in both long and short hops, whereas GAT only focuses on its one-hop neighbors.

SA-GCN (Yang et al., 2023). Structure-aware GCN (SA-GCN) introduces a new distance-aware PU loss function that heavily relies on information within the graph structure to provide more precise supervision.

Asy-Loss (Wang et al., 2021). Asy-Loss is an innovative asymmetric loss function created through minor adjustments to the traditional cross-entropy loss, without relying on class priors. Originally developed for Positive-Unlabeled (PU) learning in image data, it has been adapted for graph data by utilizing GCN as the encoder.

Self-Correct (Zhu et al., 2023). The Noise Negative Sample Self-correction (Self-Correct) technique continuously adjusts the weights of samples in the Positive-Unlabeled (PU) set. By considering unlabeled data as negative, it converts the original semi-supervised training into supervised training with weighted samples. Similar to Asy-Loss, this approach was first created for PU learning with image data and has been modified for graph data with GCN serving as the encoder.

LPPG (Carnevali et al., 2021). The Label Propagation Method (LABEL-PPG) leverages Dijkstra's algorithm to identify reliable negative nodes and then employs label propagation to classify the remaining unknown nodes.

PiCO (Wang, Xiao et al., 2022). PiCO is specifically designed to address the problem of Partial Label Learning (PLL) for image data. It includes a representation learning module based on supervised contrastive learning and a label disambiguation algorithm based on all class prototypes. The former is used to learn effective representations, providing a foundation for label disambiguation by the latter, and the disambiguated labels are used for learning a binary classifier. Additionally, PiCO utilizes the learned classifier to classify disambiguated samples, providing supervisory information for supervised contrastive learning, forming an Expectation–Maximization effect. Since PiCO was originally designed for image data, we have replaced its image encoder with a Graph Neural Network (GNN) and made significant modifications to enable it to process graph data.

Furthermore, we established fully supervised benchmarks by training models on a labeled dataset containing both the P-set and all true

negative nodes. These benchmarks included: (i) a supervised GCN implementation (denoted as S-GCN), and (ii) state-of-the-art graph node classification methods MaskGAE (Li et al., 2023) and GraphCon (Rusch, Chamberlain, Rowbottom, Mishra, & Bronstein, 2022). It is important to note that in a PU learning setting, true negatives are typically unavailable. Their inclusion here serves exclusively for the purpose of enhancing baseline performance.

(2) Datasets. Note that in real-world applications, user interests typically represent a minority of available items. For instance, in web search (WWW applications), users engage with only a small fraction of returned pages, in academic networks, researchers focus on a limited subset of publications. Our experimental design intentionally mirrors these practical scenarios through: (i) constructing imbalanced dataset distributions during preprocessing; (ii) evaluation metrics prioritizing rare-class detection (F1-score).

Specifically, we utilize nine real-life datasets obtained from various applications, including citation networks like Cora, Citeseer, and Pubmed, co-purchasing networks like Amazon Computers (Am.Comp) and Amazon Photos (Am.Photos), co-authorship networks like Coauthor CS (Co.CS) and Coauthor Physics (Co.Phy), the Wikipedia network (WikiCS), and a social network (LastFM-Asia). The summary of these datasets can be found in Table 1.

Preprocessing. We transform the mentioned datasets into the binary classification version by treating the class with the median numbers of nodes as the positive and the rest as negative. Specifically, we designate the classes with index 2, 4, 1, 7, 4, 14, 1, 5, 15 as positive for Cora, Citeseer, Pubmed, Am.Comp, Am.Photos, Co.CS, Co.Phy, WikiCS, and LastFM-Asia, respectively. This allows us to readily access the true class prior, which we then supply for the baselines that depend on it, such as GCN-PU, GAT-PU, LSDAN, GRAB, SA-GCN, and Self-Correct, throughout all our experiments.

To supply the P set and U set for all algorithms, we randomly select a proportion p of positive nodes as the labeled P set, with the remaining as the U set. If not otherwise specified, we set $p = 20\%$. Furthermore, we select $|V_T| = 3 * |V_P|$ of unlabeled node as default for training binary classifier in GCN-PU, GAT-PU, LSDAN, BLP and our BLD.

Evaluation Metric. We evaluate the effectiveness of our BLD method using the F1 score metric (Carnevali et al., 2021; Wu et al., 2021; Yang et al., 2023). In each experimental configuration, we perform 10 rounds of evaluation and present the average results.

Implementation details. In all experiments, both GNN encoders and MLPs adopt a two-hidden-layer architecture, where the first and second layers contain 32 and 16 hidden units correspondingly. For fair comparison, we additionally evaluated the three graph-specific PU learning algorithms (GRAB, LSDAN, and SA-GCN) using their original default hyperparameters. Specifically, we implemented these methods with GCN architectures containing 64, 16, and 16 hidden units respectively, denoting these baseline versions as GRAB-d, LSDAN-d, and SA-GCN-d.

All baselines use the Adam optimizer with a learning rate (lr) of 0.005 and weight decay of $5e-4$. Motivated by the proven efficacy of graph-based self-supervised learning (particularly evidenced in BGRL), our BLD framework adopts AdamW as the default optimizer, configured with a lr of $5e-4$ and weight decay of $1e-5$. For comparative analysis, we additionally evaluate BLD using the Adam optimizer under identical hyperparameter settings. In addition, in our BLD model, we set $\tau = 0.005$ for updating the online encoder and employ augmentation functions described in Thakoor et al. (2022), You et al. (2020), which involve drop-edge and drop-feature operations, to create augmented views of the graph. Specifically, we set the default probabilities for drop-edge and drop-feature for these two views to (0.3, 0.3, 0.2, 0.4).

All algorithms are implemented using PyTorch and experiments are conducted on a machine equipped with an RTX 3080 12 GB GPU. We use the source code from PyG (<https://pytorch-geometric.readthedocs.io>) for GCN and GAT, and the source code from GitHub (<https://github.com/lcq411/BLP>) for BLP. For LSDAN, GRAB, SA-GCN, LABEL-PPG,

Table 1
Dataset summary.

Name	Nodes	Edges	Features	Classes	Pos-Index	Pos-Nodes	Pos-Prior
Cora	2708	5429	1433	7	2	418	0.1544
Citeseer	3327	4732	3703	6	4	596	0.1791
Pubmed	19717	44338	500	3	1	7739	0.3925
Am.Comp	13752	245861	767	10	7	818	0.0594
Am.Photos	7650	119081	745	8	4	882	0.1153
Co.CS	18333	81894	6805	15	14	876	0.0478
Co.Phy	34493	247962	8415	5	1	5045	0.1463
WikiCS	11701	216123	300	10	5	780	0.0667
LastFM-Asia	7624	27806	7842	18	15	257	0.0337

Table 2
F1 scores on nine datasets with $p = 20\%$.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LastFM
GRAB	70.06 $\pm$ 0.83	49.00 $\pm$ 0.89	77.92 $\pm$ 0.38	8.14 $\pm$ 0.80	15.42 $\pm$ 0.99	79.92 $\pm$ 0.49	82.12 $\pm$ 1.12	9.59 $\pm$ 0.74	32.20 $\pm$ 1.15
GRAB-d	69.56 $\pm$ 0.35	48.43 $\pm$ 0.73	77.24 $\pm$ 0.76	12.37 $\pm$ 0.28	17.24 $\pm$ 0.36	79.15 $\pm$ 0.65	81.76 $\pm$ 0.15	13.26 $\pm$ 0.81	33.41 $\pm$ 0.47
BLP	88.15 $\pm$ 0.32	62.92 $\pm$ 0.43	84.19 $\pm$ 0.24	85.78 $\pm$ 0.86	91.32 $\pm$ 0.69	92.75 $\pm$ 1.03	88.22 $\pm$ 1.01	65.09 $\pm$ 0.51	65.63 $\pm$ 0.20
GCN-PU	86.14 $\pm$ 0.96	57.63 $\pm$ 0.18	80.35 $\pm$ 0.74	73.70 $\pm$ 0.37	79.42 $\pm$ 0.22	76.22 $\pm$ 0.47	83.31 $\pm$ 0.89	40.38 $\pm$ 0.10	36.69 $\pm$ 0.87
GAT-PU	84.01 $\pm$ 0.73	54.86 $\pm$ 0.79	80.98 $\pm$ 0.58	70.16 $\pm$ 0.78	86.89 $\pm$ 0.26	86.78 $\pm$ 0.71	85.82 $\pm$ 0.79	44.37 $\pm$ 0.99	49.61 $\pm$ 0.59
LSGAN	81.36 $\pm$ 0.38	53.61 $\pm$ 0.78	OOM	OOM	OOM	OOM	OOM	OOM	44.39 $\pm$ 0.27
LSGAN-d	81.46 $\pm$ 0.16	52.98 $\pm$ 0.42	OOM	OOM	OOM	OOM	OOM	OOM	45.04 $\pm$ 0.37
SA-GCN	77.28 $\pm$ 0.98	62.16 $\pm$ 1.15	82.52 $\pm$ 0.47	38.95 $\pm$ 0.32	48.15 $\pm$ 0.52	61.49 $\pm$ 0.15	OOM	20.92 $\pm$ 0.71	33.90 $\pm$ 0.55
SA-GCN-d	76.16 $\pm$ 0.29	62.38 $\pm$ 0.64	81.98 $\pm$ 0.24	40.95 $\pm$ 0.64	49.21 $\pm$ 0.34	61.03 $\pm$ 0.37	OOM	22.45 $\pm$ 0.91	32.75 $\pm$ 0.32
Asy-Loss	81.04 $\pm$ 0.59	56.86 $\pm$ 0.78	81.64 $\pm$ 0.66	73.20 $\pm$ 1.06	22.11 $\pm$ 0.72	78.23 $\pm$ 0.65	81.08 $\pm$ 1.00	29.63 $\pm$ 0.38	23.86 $\pm$ 0.52
Self-CRT	84.99 $\pm$ 0.14	63.78 $\pm$ 0.50	10.64 $\pm$ 0.22	83.06 $\pm$ 0.66	83.87 $\pm$ 1.19	77.71 $\pm$ 0.72	86.38 $\pm$ 0.92	42.69 $\pm$ 0.85	44.85 $\pm$ 1.09
LPPG	66.79 $\pm$ 1.10	60.88 $\pm$ 0.65	81.31 $\pm$ 0.20	58.45 $\pm$ 0.24	39.32 $\pm$ 0.12	26.67 $\pm$ 0.79	61.86 $\pm$ 0.18	20.23 $\pm$ 1.01	32.00 $\pm$ 0.40
GCN-UN	64.08 $\pm$ 0.32	52.95 $\pm$ 0.78	75.94 $\pm$ 0.73	76.96 $\pm$ 0.11	84.04 $\pm$ 0.52	74.75 $\pm$ 0.15	80.56 $\pm$ 0.73	26.36 $\pm$ 0.11	25.14 $\pm$ 0.49
PiCO	45.19 $\pm$ 0.14	47.63 $\pm$ 0.26	37.81 $\pm$ 0.29	51.16 $\pm$ 0.42	49.14 $\pm$ 0.27	51.16 $\pm$ 0.37	46.15 $\pm$ 0.69	39.16 $\pm$ 0.12	44.26 $\pm$ 0.35
Our BLD (AdamW)	90.23 $\pm$ 0.23	66.20 $\pm$ 0.61	84.47 $\pm$ 1.11	92.14 $\pm$ 0.12	92.62 $\pm$ 0.93	93.13 $\pm$ 0.46	89.23 $\pm$ 0.51	72.65 $\pm$ 1.13	87.68 $\pm$ 0.15
Our BLD (Adam)	90.06 $\pm$ 0.32	66.71 $\pm$ 0.35	84.13 $\pm$ 0.52	92.11 $\pm$ 0.25	91.86 $\pm$ 0.29	93.21 $\pm$ 0.58	89.17 $\pm$ 0.46	72.71 $\pm$ 0.56	87.35 $\pm$ 0.61
S-GCN	87.18 $\pm$ 1.18	65.68 $\pm$ 0.44	85.14 $\pm$ 0.13	78.29 $\pm$ 1.09	86.42 $\pm$ 0.97	82.62 $\pm$ 0.44	86.05 $\pm$ 0.36	34.96 $\pm$ 1.04	31.87 $\pm$ 0.91
MaskGAE	88.79 $\pm$ 0.76	66.79 $\pm$ 0.39	85.68 $\pm$ 0.49	83.78 $\pm$ 0.65	88.14 $\pm$ 0.24	84.28 $\pm$ 0.65	87.21 $\pm$ 1.03	37.16 $\pm$ 0.54	35.13 $\pm$ 0.28
GraphCon	88.32 $\pm$ 0.46	66.29 $\pm$ 0.51	84.91 $\pm$ 0.29	82.15 $\pm$ 0.37	87.39 $\pm$ 0.82	85.71 $\pm$ 0.49	87.36 $\pm$ 0.51	39.67 $\pm$ 0.83	41.65 $\pm$ 0.75

Asy-Loss, and Self-Correct, we implement these methods based on the descriptions provided in the original papers (Carnevali et al., 2021; Wu, Pan, Du et al., 2019; Wu et al., 2021; Yang et al., 2023; Yoo et al., 2022), as the source codes are not available.

4.2. Main results

Table 2 lists the results of all algorithms when $p = 20\%$ is applied to datasets, with OOM indicating out-of-memory errors.

Comparison of BLD with graph PU learning methods. As evidenced by Table 2, our proposed BLD demonstrates comparable performance when using either AdamW or Adam optimizers, while significantly outperforming all baseline algorithms, despite most of the baselines are provided with true class priors or true negative nodes, such as GRAB, GCN-PU, GAT-PU, LSDAN, SA-GCN, Asy-Loss, Self-Correct, and LABEL-PPG. Specifically, compared with GAT-PU, our BLD achieves improvements of 6.22%, 21.98%, 6.35%, 28.28%, and 38.07% on Cora, Am.Comp, Co.CS, WikiCS, and LastFM-Asia, respectively. Furthermore, when compared with the most recent method BLP, our BLD achieves improvements of 2.08%, 6.36%, 0.38%, 7.56%, and 22.32%, respectively.

The aforementioned baselines work either by designing complex GNN encoders to extract valuable latent embeddings, such as GAT-PU with a self-attention mechanism and LSDAN with a multi-hop attention mechanism, or by exploring useful supervised signals and designing novel loss functions like marginals of belief propagation (GRAB), prototypes (BLD), non-negative unbiased risk (GAT-PU, GCN-PU, LSDAN), and structure-aware loss (SA-GCN). In contrast, our BLD solely relied on the GCN encoder and the CE loss function.

Considering above cases, we speculate that BLD's superior performance primarily arises from its bootstrapped representation learning module and progressive label disambiguation strategy, rather than its encoder or loss function. The former effectively captures the semantic

similarity within positives and unlabeled nodes, producing representations that align with the positive class to aid label disambiguation. The latter reliably identifies ambiguous nodes based the central-region, and steadily refines their soft pseudo targets to provide reliable and robust supervision for classification learning. To gain further insights, we will conduct visual analysis on the obtained representations and updated soft pseudo targets.

Comparison of BLD with PLL methods. From Table 2, it can also be seen that the performance of our BLD significantly surpasses that of PiCO. Although PiCO strives to learn compact deep features for each class, the ambiguously labeled nodes with uniform pseudo targets are unable to offer meaningful guidance, thus failing to obtain effective prototypes of positive and negative examples, and cannot accurately perform label disambiguation. This situation demonstrates that existing PLL methods are not suitable for handling the PU problem.

Comparison of BLD with full supervised methods. Interestingly, as demonstrated in Table 2, our BLD outperforms supervised GCN (S-GCN), MaskGAE and GraphCon, which are trained with true positives and negatives in most cases. This could be attributed to that by bootstrapping previous representations and derived positive centroid as targets for of both positives and unlabeled nodes and positive, BLD can learn more useful representations. Additionally, the soft label disambiguation strategy can offer more reliable and robust supervision for classification. In comparison, those supervised models solely rely on the provided positive and negative examples for learning. These cases further highlight the effectiveness of BLD framework.

More experimental results with $p = 5\%, 10\%, 30\%, 40\%$ of positive nodes are listed in Table 3, Table 4, Table 5, and Table 6, respectively. From these tables, it can observed that in most cases, our BLD algorithm far outperforms other algorithms.

4.3. Visual analysis

Visualizing representations. We perform a visual analysis on the acquired representations to aid label disambiguation. We introduce

Table 3F1 scores on nine datasets with $p = 5\%$.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
GRAB	71.38 $\pm$ 0.59	42.07 $\pm$ 0.39	69.42 $\pm$ 1.14	8.94 $\pm$ 0.85	18.09 $\pm$ 1.18	71.28 $\pm$ 0.50	81.46 $\pm$ 0.77	8.88 $\pm$ 0.78	28.28 $\pm$ 0.27
BLP	80.60 $\pm$ 1.11	58.46 $\pm$ 0.40	80.26 $\pm$ 0.35	67.67 $\pm$ 0.16	24.03 $\pm$ 0.65	71.82 $\pm$ 0.97	83.59 $\pm$ 0.65	28.03 $\pm$ 0.49	33.10 $\pm$ 0.58
GCN-PU	86.38 $\pm$ 0.26	50.20 $\pm$ 0.99	81.99 $\pm$ 0.31	69.84 $\pm$ 0.55	81.89 $\pm$ 0.61	65.95 $\pm$ 0.53	81.50 $\pm$ 0.92	35.10 $\pm$ 0.16	38.34 $\pm$ 0.17
GAT-PU	80.09 $\pm$ 0.73	55.41 $\pm$ 1.17	80.26 $\pm$ 0.15	68.72 $\pm$ 0.40	86.04 $\pm$ 0.13	82.09 $\pm$ 0.23	84.26 $\pm$ 1.18	31.97 $\pm$ 1.14	40.65 $\pm$ 0.74
LSGAN	78.07 $\pm$ 0.75	58.70 $\pm$ 0.56	OOM	OOM	OOM	OOM	OOM	OOM	39.57 $\pm$ 0.55
SA-GCN	67.54 $\pm$ 1.11	52.01 $\pm$ 0.18	78.77 $\pm$ 1.06	46.26 $\pm$ 0.62	44.34 $\pm$ 0.42	53.16 $\pm$ 0.65	OOM	31.08 $\pm$ 1.17	42.34 $\pm$ 0.20
Asy-Loss	77.60 $\pm$ 1.07	62.39 $\pm$ 1.08	12.56 $\pm$ 1.10	76.78 $\pm$ 0.79	77.38 $\pm$ 0.88	74.87 $\pm$ 0.71	83.92 $\pm$ 0.68	36.32 $\pm$ 0.11	57.55 $\pm$ 1.04
Self-CRT	82.93 $\pm$ 0.56	60.16 $\pm$ 0.38	77.45 $\pm$ 0.12	86.45 $\pm$ 0.96	88.57 $\pm$ 0.53	90.18 $\pm$ 0.95	83.87 $\pm$ 0.39	61.78 $\pm$ 0.36	69.55 $\pm$ 0.66
LPPG	60.72 $\pm$ 0.13	53.76 $\pm$ 0.36	76.79 $\pm$ 0.29	51.09 $\pm$ 1.17	40.24 $\pm$ 0.58	23.62 $\pm$ 0.59	58.57 $\pm$ 1.10	21.32 $\pm$ 0.77	29.88 $\pm$ 0.50
GCN-UN	65.77 $\pm$ 0.20	51.52 $\pm$ 0.69	70.69 $\pm$ 0.65	75.86 $\pm$ 1.02	79.93 $\pm$ 0.44	70.20 $\pm$ 0.18	81.59 $\pm$ 1.18	28.04 $\pm$ 0.16	32.16 $\pm$ 0.64
Our BLD (AdamW)	88.35 $\pm$ 0.99	61.83 $\pm$ 1.11	83.28 $\pm$ 0.38	90.52 $\pm$ 0.23	90.68 $\pm$ 0.57	92.13 $\pm$ 0.21	88.49 $\pm$ 0.27	67.84 $\pm$ 1.10	83.56 $\pm$ 0.24

Table 4F1 scores on nine datasets with $p = 10\%$.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
GRAB	67.53 $\pm$ 1.04	49.37 $\pm$ 1.24	70.87 $\pm$ 0.92	8.29 $\pm$ 0.96	17.16 $\pm$ 0.76	76.24 $\pm$ 0.83	81.72 $\pm$ 1.33	9.29 $\pm$ 0.96	28.25 $\pm$ 0.82
BLP	77.05 $\pm$ 0.89	54.32 $\pm$ 0.75	79.64 $\pm$ 0.58	74.05 $\pm$ 0.67	23.95 $\pm$ 0.82	80.76 $\pm$ 0.53	82.39 $\pm$ 0.72	30.24 $\pm$ 0.61	35.94 $\pm$ 0.59
GCN-PU	87.28 $\pm$ 0.42	51.30 $\pm$ 0.33	83.21 $\pm$ 0.25	69.72 $\pm$ 0.48	78.40 $\pm$ 0.37	75.12 $\pm$ 0.29	81.18 $\pm$ 0.34	31.35 $\pm$ 0.41	44.02 $\pm$ 0.52
GAT-PU	85.60 $\pm$ 0.68	49.93 $\pm$ 0.77	81.58 $\pm$ 0.59	68.45 $\pm$ 0.73	83.16 $\pm$ 0.81	84.92 $\pm$ 0.64	85.96 $\pm$ 0.79	34.78 $\pm$ 0.85	41.27 $\pm$ 0.67
LSGAN	77.06 $\pm$ 0.97	49.21 $\pm$ 0.88	OOM	OOM	OOM	OOM	OOM	OOM	34.45 $\pm$ 0.49
SA-GCN	65.79 $\pm$ 0.61	54.75 $\pm$ 0.54	78.25 $\pm$ 0.48	44.69 $\pm$ 0.57	45.23 $\pm$ 0.63	59.90 $\pm$ 0.49	OOM	29.99 $\pm$ 0.56	38.87 $\pm$ 0.68
Asy-Loss	85.52 $\pm$ 0.73	55.71 $\pm$ 0.66	11.21 $\pm$ 0.29	78.05 $\pm$ 0.59	79.21 $\pm$ 0.72	80.06 $\pm$ 0.65	86.29 $\pm$ 0.77	37.58 $\pm$ 0.83	50.53 $\pm$ 0.79
Self-CRT	77.86 $\pm$ 0.85	63.29 $\pm$ 0.97	81.56 $\pm$ 0.89	80.26 $\pm$ 0.91	90.44 $\pm$ 0.84	91.89 $\pm$ 0.76	84.58 $\pm$ 0.69	63.87 $\pm$ 0.82	63.51 $\pm$ 0.90
LPPG	66.67 $\pm$ 0.78	57.02 $\pm$ 0.69	81.32 $\pm$ 0.64	57.56 $\pm$ 0.57	42.89 $\pm$ 0.68	27.86 $\pm$ 0.71	65.54 $\pm$ 0.83	26.82 $\pm$ 0.75	31.84 $\pm$ 0.87
GCN-UN	62.61 $\pm$ 0.49	51.13 $\pm$ 0.42	73.43 $\pm$ 0.38	78.52 $\pm$ 0.46	82.84 $\pm$ 0.54	79.27 $\pm$ 0.41	77.29 $\pm$ 0.37	26.43 $\pm$ 0.45	29.11 $\pm$ 0.53
Our BLD (AdamW)	90.10 $\pm$ 0.63	64.42 $\pm$ 0.57	84.24 $\pm$ 0.51	91.47 $\pm$ 0.68	91.34 $\pm$ 0.59	92.42 $\pm$ 0.71	88.83 $\pm$ 0.14	68.94 $\pm$ 0.39	86.46 $\pm$ 0.76

Table 5F1 scores on nine datasets with $p = 30\%$.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
GRAB	73.99 $\pm$ 0.65	43.06 $\pm$ 0.98	73.33 $\pm$ 1.10	7.82 $\pm$ 0.45	15.12 $\pm$ 0.78	77.91 $\pm$ 0.29	85.46 $\pm$ 0.54	13.92 $\pm$ 0.87	29.51 $\pm$ 1.02
BLP	75.52 $\pm$ 0.34	55.28 $\pm$ 0.56	80.83 $\pm$ 0.77	73.43 $\pm$ 1.15	27.33 $\pm$ 0.23	77.67 $\pm$ 0.89	79.13 $\pm$ 0.45	29.63 $\pm$ 0.67	21.30 $\pm$ 0.98
GCN-PU	80.67 $\pm$ 0.54	58.33 $\pm$ 1.02	79.88 $\pm$ 0.45	71.14 $\pm$ 0.67	77.23 $\pm$ 1.19	75.53 $\pm$ 0.34	81.12 $\pm$ 0.78	32.19 $\pm$ 0.56	39.96 $\pm$ 0.89
GAT-PU	76.08 $\pm$ 0.98	51.88 $\pm$ 0.65	79.45 $\pm$ 0.87	66.57 $\pm$ 0.34	86.18 $\pm$ 0.56	83.52 $\pm$ 1.02	82.75 $\pm$ 0.45	35.64 $\pm$ 1.10	43.39 $\pm$ 0.78
LSGAN	78.90 $\pm$ 0.45	54.79 $\pm$ 0.78	OOM	OOM	OOM	OOM	OOM	OOM	35.86 $\pm$ 0.67
SA-GCN	72.29 $\pm$ 1.02	58.21 $\pm$ 0.34	81.37 $\pm$ 0.98	40.43 $\pm$ 0.78	52.46 $\pm$ 1.10	61.91 $\pm$ 0.56	OOM	25.08 $\pm$ 0.45	31.28 $\pm$ 0.89
Asy-Loss	84.06 $\pm$ 0.78	62.14 $\pm$ 1.15	10.65 $\pm$ 0.34	81.44 $\pm$ 0.98	85.15 $\pm$ 0.56	83.15 $\pm$ 1.02	86.22 $\pm$ 0.45	44.57 $\pm$ 1.10	57.66 $\pm$ 0.78
Self-CRT	85.99 $\pm$ 0.67	59.92 $\pm$ 0.89	83.09 $\pm$ 0.54	83.46 $\pm$ 0.78	88.58 $\pm$ 0.34	90.08 $\pm$ 1.19	85.52 $\pm$ 0.56	58.05 $\pm$ 0.45	61.77 $\pm$ 0.98
LPPG	61.82 $\pm$ 1.10	60.65 $\pm$ 0.78	83.13 $\pm$ 0.45	54.26 $\pm$ 0.98	40.56 $\pm$ 1.02	31.18 $\pm$ 0.34	60.12 $\pm$ 0.78	23.18 $\pm$ 0.89	32.67 $\pm$ 0.67
GCN-UN	57.42 $\pm$ 0.56	50.38 $\pm$ 0.98	76.84 $\pm$ 0.77	80.54 $\pm$ 0.45	86.12 $\pm$ 1.15	77.50 $\pm$ 0.89	79.37 $\pm$ 0.34	25.75 $\pm$ 1.02	24.35 $\pm$ 0.78
Our BLD (AdamW)	91.08 $\pm$ 0.34	71.06 $\pm$ 0.67	76.84 $\pm$ 0.98	91.34 $\pm$ 0.56	91.22 $\pm$ 0.78	91.64 $\pm$ 0.45	88.54 $\pm$ 0.89	69.71 $\pm$ 1.10	87.22 $\pm$ 0.67

Table 6F1 scores on nine datasets with $p = 40\%$.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
GRAB	73.88 $\pm$ 1.19	45.19 $\pm$ 0.20	72.87 $\pm$ 0.27	7.99 $\pm$ 1.00	13.59 $\pm$ 0.89	73.71 $\pm$ 0.73	77.98 $\pm$ 0.77	11.81 $\pm$ 0.50	28.45 $\pm$ 0.49
BLP	74.65 $\pm$ 1.12	54.29 $\pm$ 0.39	78.37 $\pm$ 0.10	71.56 $\pm$ 0.36	20.42 $\pm$ 0.65	75.72 $\pm$ 0.78	79.77 $\pm$ 0.33	28.08 $\pm$ 1.01	21.42 $\pm$ 0.97
GCN-PU	80.07 $\pm$ 1.02	59.24 $\pm$ 1.08	76.41 $\pm$ 0.28	66.35 $\pm$ 0.68	72.88 $\pm$ 1.07	74.45 $\pm$ 0.53	78.39 $\pm$ 0.78	32.08 $\pm$ 0.62	34.78 $\pm$ 0.62
GAT-PU	75.76 $\pm$ 0.12	51.64 $\pm$ 1.03	74.27 $\pm$ 0.99	70.02 $\pm$ 0.71	85.24 $\pm$ 1.10	84.26 $\pm$ 0.80	85.66 $\pm$ 0.37	34.89 $\pm$ 0.34	47.15 $\pm$ 0.82
LSGAN	80.71 $\pm$ 1.04	48.17 $\pm$ 0.77	OOM	OOM	OOM	OOM	OOM	OOM	40.13 $\pm$ 0.86
SA-GCN	67.67 $\pm$ 0.42	54.97 $\pm$ 0.50	76.28 $\pm$ 0.89	38.26 $\pm$ 0.73	51.41 $\pm$ 1.09	59.17 $\pm$ 0.25	OOM	21.86 $\pm$ 0.76	27.25 $\pm$ 0.55
Asy-Loss	76.22 $\pm$ 0.42	60.24 $\pm$ 0.31	13.26 $\pm$ 1.07	81.05 $\pm$ 0.52	78.61 $\pm$ 0.16	79.63 $\pm$ 0.44	83.24 $\pm$ 0.95	35.16 $\pm$ 0.47	59.93 $\pm$ 0.76
Self-CRT	86.78 $\pm$ 0.75	58.89 $\pm$ 0.72	83.26 $\pm$ 0.41	80.99 $\pm$ 0.90	86.78 $\pm$ 0.18	87.18 $\pm$ 0.43	87.76 $\pm$ 0.92	53.41 $\pm$ 0.14	64.78 $\pm$ 0.54
LPPG	60.78 $\pm$ 1.02	59.58 $\pm$ 0.29	82.21 $\pm$ 0.45	53.75 $\pm$ 0.41	39.35 $\pm$ 0.55	30.54 $\pm$ 0.67	58.88 $\pm$ 0.89	22.08 $\pm$ 0.56	30.45 $\pm$ 0.78
GCN-UN	65.33 $\pm$ 0.45	46.98 $\pm$ 0.67	77.65 $\pm$ 0.34	79.46 $\pm$ 0.92	82.68 $\pm$ 0.78	79.81 $\pm$ 0.56	78.29 $\pm$ 0.45	26.08 $\pm$ 1.01	28.45 $\pm$ 0.89
Our BLD (AdamW)	91.86 $\pm$ 0.34	70.34 $\pm$ 0.67	76.74 $\pm$ 0.92	91.67 $\pm$ 0.45	90.56 $\pm$ 0.78	90.62 $\pm$ 0.56	88.15 $\pm$ 0.89	68.67 $\pm$ 0.78	86.34 $\pm$ 0.92

two variants of BLD for comparison. One variant solely employs bootstrapped self-supervised learning task (BLD-S), while the other utilizes exclusively centroid-supervised (BLD-C). We employ the t-SNE embedding tool for visualization (der Maaten & Hinton, 2008). Fig. 3 displays the visualized representations generated by BLD-C, BLD-S, and BLD on Cora, Co.CS, and Am.Photos when training convergence.

From Fig. 3, it is interesting to note that BLD-C does not collapse to trivial solutions, although only positive nodes are used for training. This could be attributed to the positive centroid constantly moving in each epoch, preventing the loss from reaching zero. However, neither BLD-C nor BLD-S produces closely aligned representations for the positive and negative classes and lacks a clear distinction between them. In

contrast, BLD successfully pulls positive nodes closer and occupying a smaller region, while negative nodes are uniformly distributed outside this region. This can be attributed to the fact that by combining both learning tasks, BLD can simultaneously leverage low-level semantic similarity to help bring positives closer and utilize uniformity to separate negatives. Based on such representations, our central region-based label disambiguation strategy successfully identifies the ambiguously labeled nodes, as shown in Fig. 3.

Visualizing soft pseudo target. We plot the curve of F1-scores directly indicated by the soft pseudo targets for the ambiguously labeled nodes throughout the training. Additionally, we compare this curve with the F1-scores indicated by the central region for these nodes, as

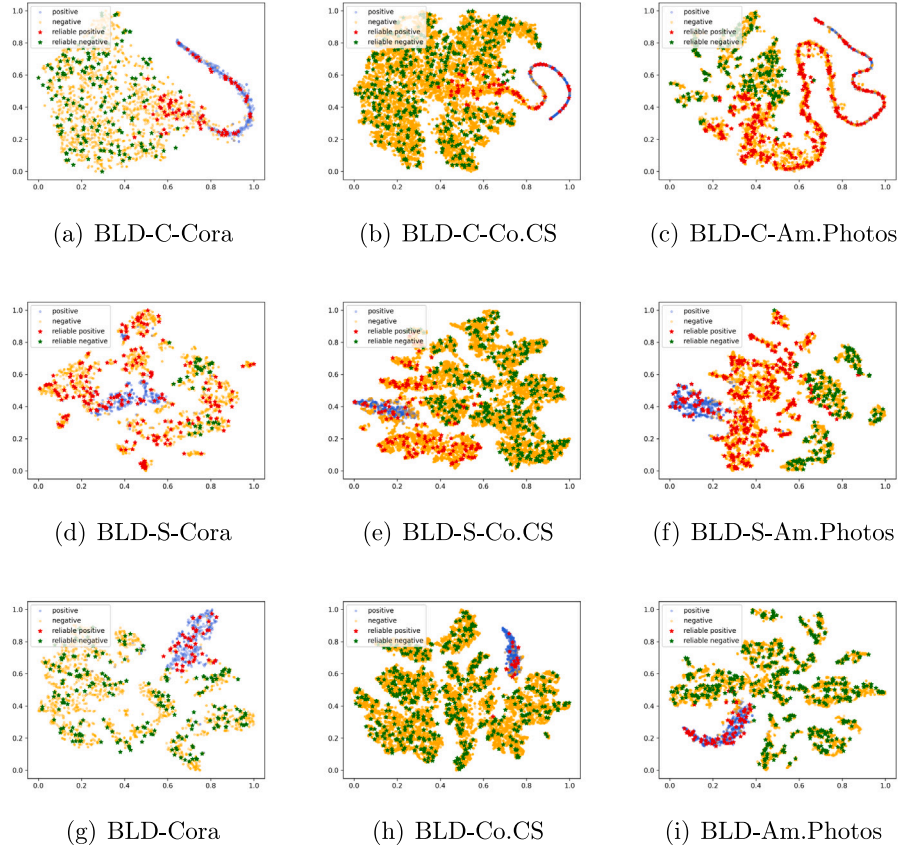


Fig. 3. T-SNE visualization of produced representations. Blue represents positive nodes, orange denotes negative nodes, red indicates reliable positives identified by the central region-based strategy from ambiguously labeled nodes, and green represents identified reliable negatives, covering positive and negative regions, respectively, with very few identification errors.

well as the F1-scores obtained by predicting on all unlabeled nodes using the binary classifier. We analyze their correlated trends to verify the effectiveness of the central region-based disambiguation strategy and refer to these three components as the soft-label, central-region, and binary-classifier, respectively. Fig. 4 displays the results for Cora, Co.CS, and Am.Photos datasets.

From Fig. 4, we can be observed that as the training epochs increase, the F1-scores of all three components rise as well. Central-region initially increase, followed by soft-label, and lastly the binary-classifier. This trend occurs because with more training rounds, enhanced representations are learned, supporting the central-region to better discerning whether ambiguously labeled nodes belong inside or outside the region. The improved determinations are then progressively reflected in soft pseudo targets, subsequently affecting in the binary classifier. Once a certain threshold is reached, all three components maintain relatively stable performance. However, the central-region exhibits significant fluctuations, while the soft-label remains more stable due to the progressive updating mechanism, and the binary classifier is the most stable. These observations indicate that our disambiguation strategy offers reliable and robust supervised information for training the binary classifier.

4.4. Ablation study

Effect of representation learning tasks. We explore the effects of two bootstrapped learning tasks, centroid-supervised and self-supervised, on classification performance. Consistent with our earlier discussion, we denote these tasks exclusively used our BLD framework as BLD-C and BLD-S. The results for Cora, Am.Comp, Am.Photos, Co.CS, and Co.Phy are presented in Table 7. It is evident that BLD-C avoids

trivial solutions for reasons previously discussed. However, the classification effectiveness of BLD notably surpasses that of each individual task. This finding suggests that both learning tasks contribute to the overall classification performance, with their combination leading to a remarkable enhancement.

Effect of back-propagate classification loss. We investigate the impact of back-propagated classification learning loss on updating the encoder towards the final classification performance. Specifically, we backpropagate the label prediction loss calculated by Eq. (1) to the encoder f_θ , involving it in the update of its parameters. For ease of discussion, we term the BLD employing this method as BLD-BPCL. The results presented in Table 8 show that the performance is comparable in terms of classification loss with and without back-propagation, both exhibiting excellent performance, highlighting the BLD framework's capability to merge classification learning and representation learning. However, when back-propagation is applied, the performance is slightly inferior compared to when it is not used. This could potentially be due to the fact that label disambiguation introduces some errors into the soft targets of unlabeled nodes, which in turn affects representation learning and ultimately the classification performance.

Effect of unbiased loss vs. disambiguation loss. We investigate the effectiveness of utilizing disambiguated labels (soft pseudo targets) for supervised classification learning. For comparison, we replace the Cross-Entropy (CE) loss in BLD with a non-negative unbiased loss (Kiryo et al., 2017) that relies on the class prior, enabling direct learning from PU nodes. By either back-propagating or not back-propagating this loss to update the main encoder, we generate two BLD variants: BLD-T-UL and BLD-F-UL. The results from BLD and these two variants are presented in Table 9. It can be observed that BLD significantly outperforms the both two variants. These results demonstrate the effectiveness of using disambiguated labels for supervision.

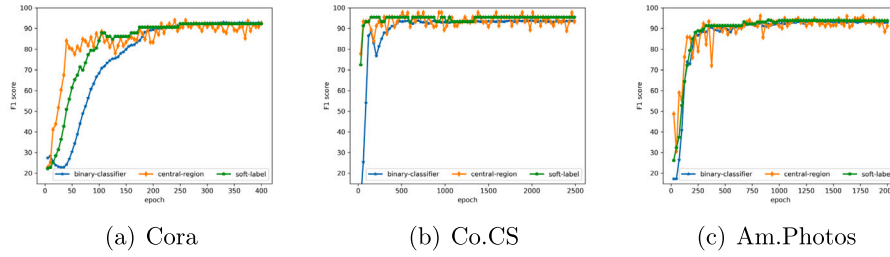


Fig. 4. Visualization the F1 scores indicated by central-region, soft-label, and binary-classifier.

Table 7

Effect of bootstrapped learning tasks.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
BLD-C	87.84 $\pm$ 0.85	62.18 $\pm$ 0.85	56.37 $\pm$ 0.85	46.87 $\pm$ 0.53	53.48 $\pm$ 0.52	87.22 $\pm$ 0.38	83.79 $\pm$ 0.83	24.29 $\pm$ 0.79	37.67 $\pm$ 0.71
BLD-S	33.49 $\pm$ 0.40	25.98 $\pm$ 0.59	50.83 $\pm$ 0.67	28.33 $\pm$ 0.97	34.46 $\pm$ 0.73	25.10 $\pm$ 0.35	25.14 $\pm$ 0.71	14.51 $\pm$ 0.94	15.31 $\pm$ 0.56
BLD	90.23 $\pm$ 0.23	66.20 $\pm$ 0.61	84.47 $\pm$ 1.11	92.14 $\pm$ 0.12	92.62 $\pm$ 0.93	93.13 $\pm$ 0.46	89.23 $\pm$ 0.51	72.65 $\pm$ 1.13	87.68 $\pm$ 0.15

Table 8

Effect of back-propagate classification loss.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
BLD-BPCL	88.79 $\pm$ 1.04	64.12 $\pm$ 0.34	82.26 $\pm$ 0.39	90.56 $\pm$ 0.62	90.36 $\pm$ 0.34	91.24 $\pm$ 0.50	87.22 $\pm$ 1.11	70.64 $\pm$ 0.58	85.19 $\pm$ 0.54
BLD	90.23 $\pm$ 0.23	66.20 $\pm$ 0.61	84.47 $\pm$ 1.11	92.14 $\pm$ 0.12	92.62 $\pm$ 0.93	93.13 $\pm$ 0.46	89.23 $\pm$ 0.51	72.65 $\pm$ 1.13	87.68 $\pm$ 0.15

Table 9

Effect of unbiased loss vs. disambiguation loss.

Type	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
BLD-T-UL	60.25 $\pm$ 0.75	50.16 $\pm$ 0.95	67.16 $\pm$ 0.81	71.04 $\pm$ 0.34	68.88 $\pm$ 1.03	57.30 $\pm$ 1.15	82.03 $\pm$ 0.55	33.87 $\pm$ 0.56	31.16 $\pm$ 0.52
BLD-F-UL	65.95 $\pm$ 0.91	53.68 $\pm$ 0.63	72.92 $\pm$ 0.62	76.32 $\pm$ 0.86	76.10 $\pm$ 0.36	56.48 $\pm$ 1.14	83.63 $\pm$ 0.60	36.15 $\pm$ 0.82	34.38 $\pm$ 0.34
BLD	90.23 $\pm$ 0.23	66.20 $\pm$ 0.61	84.47 $\pm$ 1.11	92.14 $\pm$ 0.12	92.62 $\pm$ 0.93	93.13 $\pm$ 0.46	89.23 $\pm$ 0.51	72.65 $\pm$ 1.13	87.68 $\pm$ 0.15

Table 10

Effect of diverse γ .

γ	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
0.99	90.23 $\pm$ 0.23	66.20 $\pm$ 0.61	84.47 $\pm$ 1.11	92.14 $\pm$ 0.12	92.62 $\pm$ 0.93	93.13 $\pm$ 0.46	89.23 $\pm$ 0.51	72.65 $\pm$ 1.13	87.68 $\pm$ 0.15
0.98	90.71 $\pm$ 1.00	66.34 $\pm$ 0.64	85.02 $\pm$ 0.65	92.68 $\pm$ 0.29	92.41 $\pm$ 0.60	93.23 $\pm$ 0.73	89.90 $\pm$ 0.10	72.24 $\pm$ 0.65	87.35 $\pm$ 0.35
0.97	90.95 $\pm$ 0.66	66.12 $\pm$ 0.94	84.67 $\pm$ 0.32	92.31 $\pm$ 1.17	92.08 $\pm$ 1.01	93.71 $\pm$ 0.19	90.06 $\pm$ 0.58	72.67 $\pm$ 0.32	87.54 $\pm$ 1.05
0.96	91.03 $\pm$ 0.80	66.25 $\pm$ 0.63	84.34 $\pm$ 1.03	92.23 $\pm$ 0.92	91.89 $\pm$ 0.55	93.09 $\pm$ 0.35	89.70 $\pm$ 1.03	72.54 $\pm$ 0.35	87.46 $\pm$ 0.96
0.95	91.01 $\pm$ 0.76	65.99 $\pm$ 0.42	84.41 $\pm$ 0.94	92.06 $\pm$ 0.11	92.13 $\pm$ 0.54	93.44 $\pm$ 0.89	89.40 $\pm$ 0.68	72.69 $\pm$ 0.36	87.53 $\pm$ 0.43
0.94	90.31 $\pm$ 0.50	66.12 $\pm$ 0.18	84.62 $\pm$ 0.34	92.84 $\pm$ 0.61	92.08 $\pm$ 0.36	92.67 $\pm$ 0.54	89.99 $\pm$ 0.16	73.02 $\pm$ 0.94	88.12 $\pm$ 0.61
0.93	89.98 $\pm$ 0.36	65.91 $\pm$ 0.31	84.34 $\pm$ 0.61	92.89 $\pm$ 0.55	92.54 $\pm$ 1.19	93.42 $\pm$ 0.26	90.07 $\pm$ 0.50	72.76 $\pm$ 0.63	87.92 $\pm$ 0.31
0.92	91.05 $\pm$ 0.65	67.01 $\pm$ 0.21	84.31 $\pm$ 1.12	92.99 $\pm$ 0.68	92.62 $\pm$ 0.85	92.68 $\pm$ 1.02	89.71 $\pm$ 0.24	72.82 $\pm$ 0.34	87.41 $\pm$ 0.38
0.91	90.16 $\pm$ 0.33	66.56 $\pm$ 0.39	84.52 $\pm$ 0.26	92.52 $\pm$ 0.45	92.77 $\pm$ 0.40	92.74 $\pm$ 0.98	89.89 $\pm$ 1.03	73.11 $\pm$ 1.02	87.82 $\pm$ 0.82
0.90	90.91 $\pm$ 0.23	66.47 $\pm$ 0.51	84.42 $\pm$ 0.85	92.08 $\pm$ 0.40	92.35 $\pm$ 1.04	93.20 $\pm$ 0.98	89.76 $\pm$ 0.60	72.57 $\pm$ 0.63	87.72 $\pm$ 0.45

Effect of γ for label disambiguation. We examine the influence of decay rates γ on updating soft pseudo targets on the ultimate classification performance. Results presented in Table 10 reveal that the classification performance of BLD remains consistent across various datasets with varying decay rates, showing the robustness of our BLD model to this hyper-parameter.

Effect of the number of V_T . We examine how selecting different quantities of unlabeled nodes V_T for training the classifier affects the final classification performance. Table 11 presents the results as the numbers span from $k = 1$ to $k = 5$ times the number of positive nodes. The performance appears consistent across different quantities of unlabeled nodes, highlighting the robustness of BLD to the variations in numbers of V_T .

Effect of reliable positives for representation learning. Note that the trained classifier can be utilized to predict the label of unlabeled nodes, we employ it to select the most reliable positive nodes from V_T and add them to P set for representation learning in our BLD. Specifically, we employ the classifier trained in the previous step to identify unlabeled categories, selecting nodes with the highest predicted probability of being positive as reliable positive nodes, which are then added to the P-set. We refer to such approach as RL-POS

and evaluate its impact on the final classification performance. The experimental results are presented in Table 12. It can be observed that the inclusion or exclusion of reliable positives yields similar results in most cases, but shows a relatively significant decrease on the Pubmed and WikiCS datasets. This suggests that our representation learning algorithm has already captured as much similarity information of the nodes as possible, and the addition of reliable positives does not necessarily enhance the representation. Moreover, in a few instances, there is a possibility of introducing false positives, which may accumulate in the representation and classifier, leading to a decrease in classification performance.

Effect of augmentation. We explore the impact of augmentation on classification performance by setting two augmented operations—drop-edge and drop-feature—with varying probabilities to generate two views. Table 13 displays the classification performance across different sets of dropping probabilities (DropProb). In the 4-tuple listed in the first column of the table, the first and second entries represent the edge-dropping probability and feature-dropping probability for the first augmentation, respectively, while the third and fourth entries correspond to the edge-dropping probability and feature-dropping probability for the second augmentation. Our findings indicate that the

Table 11Effect of various number of V_T .

k	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
1	90.59 ± 0.53	66.37 ± 0.52	84.28 ± 0.81	92.86 ± 0.77	92.21 ± 0.65	93.95 ± 0.39	89.80 ± 1.04	72.73 ± 0.67	87.35 ± 0.63
2	90.45 ± 0.74	66.45 ± 0.34	84.65 ± 0.64	93.05 ± 1.05	93.01 ± 0.75	93.66 ± 0.36	90.13 ± 0.20	72.42 ± 0.61	87.46 ± 0.47
3	90.23 ± 0.23	66.20 ± 0.61	84.47 ± 1.11	92.14 ± 0.12	92.62 ± 0.93	93.13 ± 0.46	89.23 ± 0.51	72.65 ± 1.13	87.68 ± 0.15
4	90.25 ± 0.25	66.76 ± 0.98	84.82 ± 0.95	92.89 ± 0.42	92.95 ± 1.19	93.20 ± 0.31	90.08 ± 0.93	72.72 ± 0.87	87.91 ± 0.94
5	90.19 ± 1.10	66.32 ± 0.71	84.73 ± 0.39	92.59 ± 0.33	93.18 ± 0.90	93.58 ± 0.94	89.35 ± 1.17	72.37 ± 0.93	87.42 ± 0.39

Table 12

Effect of reliable positives for representation learning.

Methods	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
RL-POS	91.27 ± 1.03	68.77 ± 0.17	77.60 ± 0.82	92.13 ± 0.34	91.82 ± 1.13	92.09 ± 1.10	89.28 ± 0.46	64.20 ± 0.23	87.27 ± 0.74
our BLD	90.23 ± 0.23	66.20 ± 0.61	84.47 ± 1.11	92.14 ± 0.12	92.62 ± 0.93	93.13 ± 0.46	89.23 ± 0.51	72.65 ± 1.13	87.68 ± 0.15

Table 13

Effect of augmentation.

DropProb	Cora	Citeseer	Pubmed	Am.Comp	Am.Photos	Co.CS	Co.Phy	WikiCS	LasfTM
(0.3, 0.3, 0.2, 0.4)	90.23 ± 0.23	66.20 ± 0.61	84.47 ± 1.11	92.14 ± 0.12	92.62 ± 0.93	93.13 ± 0.46	89.23 ± 0.51	72.65 ± 1.13	87.68 ± 0.15
(0.4, 0.1, 0.1, 0.4)	90.27 ± 0.46	66.77 ± 0.12	84.60 ± 0.18	92.13 ± 0.59	91.82 ± 1.15	92.09 ± 0.57	89.28 ± 1.08	72.20 ± 0.86	87.27 ± 0.72
(0.5, 0.2, 0.4, 0.1)	90.87 ± 0.34	65.41 ± 0.89	84.33 ± 0.71	92.61 ± 0.27	91.87 ± 1.10	92.61 ± 0.57	89.42 ± 0.50	73.49 ± 0.79	87.15 ± 0.66
(0.2, 0.2, 0.3, 0.1)	91.02 ± 0.99	65.85 ± 0.95	84.24 ± 0.53	91.46 ± 1.20	92.08 ± 0.97	92.55 ± 0.74	89.73 ± 0.35	73.31 ± 0.62	87.37 ± 1.10
(0.3, 0.2, 0.3, 0.1)	90.50 ± 0.13	66.96 ± 0.30	85.35 ± 0.75	91.52 ± 0.68	93.02 ± 0.85	92.70 ± 0.84	88.46 ± 1.11	72.00 ± 1.11	87.53 ± 0.46
(0.2, 0.1, 0.2, 0.2)	89.80 ± 0.57	65.76 ± 0.14	83.50 ± 0.31	92.84 ± 0.60	91.65 ± 0.48	92.79 ± 0.68	89.01 ± 0.27	72.54 ± 0.86	87.96 ± 1.05
(0.4, 0.2, 0.3, 0.2)	89.85 ± 0.16	66.99 ± 0.32	83.95 ± 0.28	92.00 ± 0.93	92.64 ± 0.12	94.02 ± 1.19	89.32 ± 1.18	71.90 ± 0.90	87.99 ± 1.20
(0.2, 0.3, 0.1, 0.3)	90.84 ± 0.33	66.50 ± 0.28	85.22 ± 0.58	92.41 ± 0.80	92.95 ± 0.90	92.85 ± 0.41	89.61 ± 0.22	73.03 ± 0.10	87.30 ± 0.73

performance remains stable despite the variation in probabilities. This suggests that while augmented views of unlabeled data are important, BLD does not rely heavily on any specific augmentation operation.

5. Conclusion

In this paper, we formulate graph PU learning as a PLL problem, where unlabeled nodes are viewed as samples ambiguously labeled as both positive and negative. We introduce the BLD framework to address this PLL problem. BLD consists of a bootstrapped representation learning module and a novel label disambiguation method. For learning module, we utilize previous representations and the derived positive centroid as targets for all nodes and positive nodes, and learn enhanced representations. This strategy eliminates the requirement for a class prior, leading to closely-aligned and distinct representations for positive nodes. For label disambiguation, we employ the obtained representations and the positive centroid to define a reliable region for resolving label ambiguity. Experimental results on various datasets show that our BLD framework surpasses state-of-the-art methods and even outperforms supervised GNN algorithms in most cases. However, the proposed method in this study is currently limited to homogeneous graphs and does not address heterogeneous graphs. In future work, we plan to extend our approach to tackle PU learning problems on heterogeneous graphs.

CRedit authorship contribution statement

Chunquan Liang: Writing – original draft, Methodology, Conceptualization. **Luyue Wang:** Software, Investigation. **Xinyuan Feng:** Validation, Investigation. **Yuying Cheng:** Validation, Software. **Mei Li:** Writing – review & editing, Resources. **Shirui Pan:** Writing – review & editing, Methodology. **Hongming Zhang:** Supervision, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

Statement: During the preparation of this work the authors used AI research assistant in order to improve language. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is substantially supported by Science and Technology Project of the Ministry of Agriculture and Rural Affairs, National Natural Science Foundation of China (grant numbers 42377341), and Key Research and Development Program of Shaanxi Province, China (grant numbers 2024NC-ZDCYL-05-06 and 2024NC-ZDCYL-05-11).

References

- Bekker, J., & Davis, J. (2018). Learning from positive and unlabeled data: A survey. *Machine Learning*, 109, 719–760.
- Carnevali, J. C., Geraldini Rossi, R., Milios, E., & de Andrade Lopes, A. (2021). A graph-based approach for positive and unlabeled learning. *Information Sciences*, 580, 655–672.
- Chen, X., Chen, W., Chen, T., Yuan, Y., Gong, C., Chen, K., et al. (2020). Self-PU: Self boosted and calibrated positive-unlabeled training. In *Proceedings of the 37th international conference on machine learning: Vol. 119*, (pp. 1510–1519).
- Claesen, M., De Smet, F., Suykens, J. A. K., & De Moor, B. (2015). A robust ensemble approach to learn from positive and unlabeled data using SVM base models. *Neurocomputing*, 160, 73–84.
- Cour, T., Sapp, B., & Taskar, B. (2011). Learning from partial labels. *Journal of Machine Learning Research*, 12, 1501–1536.
- Duan, W., Lu, J., Wang, Y. G., & Xuan, J. (2024). Layer-diverse negative sampling for graph neural networks. *Transactions on Machine Learning Research*, 2024.
- Duan, W., Xuan, J., Qiao, M., & Lu, J. (2024). Graph convolutional neural networks with diverse negative samples via decomposed determinant point processes. *IEEE Transactions on Neural Networks and Learning Systems*, 35(12), 18160–18171.
- Grill, J. B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., et al. (2020). Bootstrap your own latent: A new approach to self-supervised learning. *Neural Information Processing Systems*.
- Guo, D., Pires, B. A., Piot, B., Grill, J. B., Altché, F., Munos, R., et al. (2020). Bootstrap latent-predictive representations for multitask reinforcement learning. In *International conference on machine learning*.
- He, J., Zhang, Y., Li, X., & Shi, P. (2012). Learning naive Bayes classifiers from positive and unlabelled examples with uncertainty. *International Journal of Systems Science*, 43, 1805–1825.
- Jain, S., Delano, J., Sharma, H., & Radivojac, P. (2020). Class prior estimation with biased positives and unlabeled examples. In *National conference on artificial intelligence*.

- Jain, S., White, M., & Radivojac, P. (2016). Estimating the class prior and posterior from noisy positives and unlabeled data. In *International conference on neural information processing systems* (pp. 2693–2701).
- Janez-Martino, F., Alaiz-Rodriguez, R., Gonzalez-Castro, V., & Fidalgo, E. (2021). Trustworthiness of spam email addresses using machine learning. In *Proceedings of the 21st ACM symposium on document engineering* (pp. 1–4).
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*.
- Kiryo, R., Niu, G., Plessis, M. C. d., & Sugiyama, M. (2017). Positive-unlabeled learning with non-negative risk estimator. In *Advances in neural information processing systems*.
- Kloumann, I. M., & Kleinberg, J. M. (2014). Community membership identification from small seed sets. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data* (pp. 1366–1375).
- Letouzey, F. (2005). *Learning from positive and unlabeled examples* (pp. 70–83). Springer Berlin Heidelberg.
- Li, P., Chien, E., Milenkovic, P., Chien, E., & Milenkovic, O. (2019). Optimizing generalized PageRank methods for seed-expansion community detection. In *Proceedings of the 33rd international conference on neural information processing systems* (pp. 11710–11721).
- Li, M., Micheli, A., Wang, Y. G., Pan, S., Lió, P., Gnecco, G. S., et al. (2024). Deep neural networks for graphs: Theory, models, algorithms, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4), 4367–4372.
- Li, J., Wu, R., Sun, W., Chen, L., Tian, S., Zhu, L., et al. (2023). What's behind the mask: Understanding masked graph modeling for graph autoencoders. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 1268–1279).
- Li, J., Zhou, P., Xiong, C., Socher, R., & Hoi, S. C. H. (2021). Prototypical contrastive learning of unsupervised representations. In *International conference on learning representations*.
- Liang, C., Tian, Y., Zhao, D., Li, M., Pan, S., Zhang, H., et al. (2024). Bootstrap latent prototypes for graph positive-unlabeled learning. *Information Fusion*, 112.
- Liu, B., Dai, Y., Li, X., Lee, W., & Yu, P. (2003). Building text classifiers using positive and unlabeled examples. In *Third IEEE international conference on data mining* (pp. 179–186).
- Liu, L., & Dietterich, T. G. (2012). A conditional multinomial mixture model for superset label learning. In *Advances in neural information processing systems* (pp. 557–565).
- Liu, L., & Peng, T. (2014). Clustering-based method for positive and unlabeled text categorization enhanced by improved TFIDF. *Journal of Information Science and Engineering*, 30, 1463–1481.
- Lv, J., Xu, M., Feng, L., Niu, G., Geng, X., & Sugiyama, M. (2020). Progressive identification of true labels for partial-label learning. In *Proceedings of the 37th international conference on machine learning*.
- Lyu, G., Feng, S., Wang, T., Lang, C., & Li, Y. (2021). GM-PLL: Graph matching based partial label learning. *IEEE Transactions on Knowledge and Data Engineering*, 33(2), 521–535.
- der Maaten, L. V., & Hinton, G. (2008). Visualizing data using t-SNE. In *International conference on machine learning*.
- Mordelet, F., & Vert, J. P. (2014). A bagging SVM to learn from positive and unlabeled examples. *Pattern Recognition Letters*, 37(feb.1), 201–209.
- Perini, L., Vercauteren, V., & Davis, J. (2020). Class prior estimation in active positive and unlabeled learning. In *International joint conference on artificial intelligence*.
- Plessis, M. C. d., Niu, G., & Sugiyama, M. (2015a). Analysis of learning from positive and unlabeled data. In *Advances in neural information processing systems*.
- Plessis, M. C. d., Niu, G., & Sugiyama, M. (2015b). Convex formulation for learning from positive and unlabeled data. In *International conference on machine learning* (pp. 1386–1394).
- Rusch, T. K., Chamberlain, B., Rowbottom, J., Mishra, S., & Bronstein, M. (2022). Graph-coupled oscillator networks. In *Proceedings of machine learning research: Vol. 162, Proceedings of the 39th international conference on machine learning* (pp. 18888–18909).
- Thakoor, S., Tallec, C., Azar, M. G., Azabou, M., Dyer, E. L., Munos, R., et al. (2022). Large-scale representation learning on graphs via bootstrapping. In *International conference on learning representations*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations*.
- Wang, S., Ding, K., Zhang, C., Chen, C., & Li, J. (2022). Task-adaptive few-shot node classification. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*.
- Wang, Y., Huang, C., Li, M., Huang, Q., Wu, X., & Wu, J. (2024). AG-meta: Adaptive graph meta-learning via representation consistency over local subgraphs. *Pattern Recognition*, 151, Article 110387.
- Wang, T., & Isola, P. (2020). Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *International conference on machine learning*.
- Wang, C., Pu, J., Xu, Z., & Zhang, J. (2021). Asymmetric loss for positive-unlabeled learning. In *2021 IEEE international conference on multimedia and expo* (pp. 1–6).
- Wang, H., Qiang, Y., Chen, C., Liu, W., Hu, T., Li, Z., et al. (2020). Online partial label learning. In *Machine learning and knowledge discovery in databases: Vol. 12458*, (pp. 455–470).
- Wang, S., Tan, Z., Liu, H., & Li, J. (2023). Contrastive meta-learning for few-shot node classification. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*.
- Wang, H., Xiao, R., Li, Y., Feng, L., Niu, G., Chen, G., et al. (2022). PiCO: Contrastive label disambiguation for partial label learning. In *International conference on learning representations*.
- Wen, H., Cui, J., Hang, H., Liu, J., Wang, Y., & Lin, Z. (2021). Leveraged weighted loss for partial label learning. Vol. 139, In *Proceedings of the 38th international conference on machine learning* (pp. 11091–11100).
- Wen, Y., Zhang, K., Li, Z., & Qiao, Y. (2016). A discriminative feature learning approach for deep face recognition. In *European conference on computer vision: Vol. 9911*, (pp. 499–515).
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2019). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*.
- Wu, M., Pan, S., Du, L., Tsang, I., Zhu, X., & Du, B. (2019). Long-short distance aggregation networks for positive unlabeled graph learning. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 2157–2160).
- Wu, M., Pan, S., Du, L., & Zhu, X. (2021). Learning graph neural networks with positive and unlabeled nodes. *ACM Transactions on Knowledge Discovery from Data*, 1–25.
- Wu, J., Pan, S., Zhu, X., Zhang, C., & Wu, X. (2016). Positive and unlabeled multi-graph learning. *IEEE Transactions on Cybernetics*, 47, <http://dx.doi.org/10.1109/TCYB.2016.2527239>.
- Xu, N., Lv, J., & Geng, X. (2019). Partial label learning via label enhancement. In *The thirty-third AAAI conference on artificial intelligence* (pp. 5557–5564).
- Yang, H., Zhang, Y., Yao, Q., & Kwok, J. (2023). Positive-unlabeled node classification with structure-aware graph learning. In *International conference on information and knowledge management* (pp. 4390–4394).
- Yoo, J., Kim, J., Yoon, H., Kim, G., Jang, C., & Kang, U. (2022). Graph-based PU learning for binary and multiclass classification without class prior. *Knowledge and Information Systems*, 64(8), 2141–2169.
- You, Y., Chen, T., Sui, Y., Chen, T., Wang, Z., & Shen, Y. (2020). Graph contrastive learning with augmentations. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems: Vol. 33*, (pp. 5812–5823).
- Zeiberg, D., Jain, S., & Radivojac, P. (2020). Fast nonparametric estimation of class proportions in the positive-unlabeled classification setting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(4), 6729–6736.
- Zhang, C., Ren, D., Liu, T., Yang, J., & Gong, C. (2019). Positive and unlabeled learning with label disambiguation. In *International joint conferences on artificial intelligence* (pp. 4250–4256).
- Zhang, M., & Yu, F. (2015). Solving the partial label learning problem: An instance-based approach. In *Proceedings of the twenty-fourth international joint conference on artificial intelligence* (pp. 4048–4054).
- Zhao, X., Chen, X., Gong, Z., Yao, W., Zhang, Y., & Zheng, X. (2022). Contrastive enhancement using latent prototype for few-shot segmentation. *arXiv e-Prints*.
- Zhao, Y., Kong, X., & Yu, P. S. (2012). Positive and unlabeled learning for graph classification. In *IEEE international conference on data mining* (pp. 962–971).
- Zhao, Y., Xu, Q., Jiang, Y., Wen, P., & Huang, Q. (2022). Dist-PU: Positive-unlabeled learning from a label distribution perspective. In *2022 IEEE/CVF conference on computer vision and pattern recognition* (pp. 14441–14450).
- Zhu, Z., Wang, L., Zhao, P., Du, C., Zhang, W., Dong, H., et al. (2023). Robust positive-unlabeled learning via noise negative sample self-correction. In *Proceedings of the 14th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 3663–3673).